# Programación

Nombre: _____ Fecha: _____

## Qué vas a aprender

- Representar un algoritmo con un diagrama de flujo
- Usar variables, condicionales, bucles y funciones
- Programar una placa microcontroladora para leer sensores y controlar actuadores
- Usar bucles, condiciones compuestas y operadores en programas
- Probar y depurar programas que controlan una placa con sensores y actuadores
- Usar vectores, else if, switch y millis() en programas para placas
- Comunicar la placa con el ordenador y desarrollar un proyecto completo

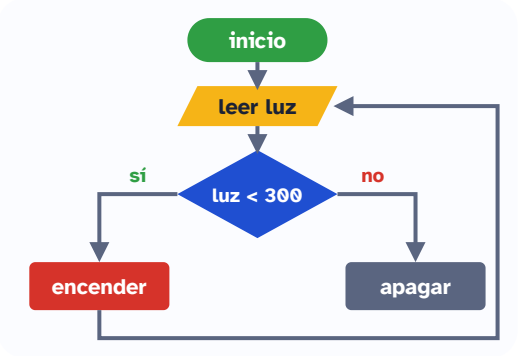
## Para pensar

**Las farolas de tu calle se encienden solas al anochecer. Si tuvieras que darles las instrucciones, ¿qué les dirías?**

## Explicación

### 1.1 Algoritmos y diagramas de flujo

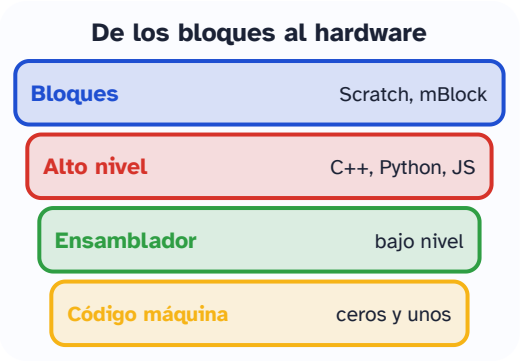
Un **algoritmo** es una secuencia ordenada y finita de pasos para resolver un problema. Antes de programar conviene dibujarlo con un **diagrama de flujo**: óvalos para el **inicio** y el **fin**, rectángulos para las **acciones**, rombos para las decisiones (sí/no) y paralelogramos para la **entrada y salida** de datos. Las flechas indican el orden.




### 1.2 Lenguajes de programación

El procesador solo entiende el **código máquina**: secuencias de ceros y unos que indican operaciones muy simples. Escribir así sería larguísimo, así que usamos **lenguajes de programación**. Los de **bajo nivel**, como el ensamblador, están muy cerca del hardware: son rápidos, pero difíciles de leer. Los de **alto nivel**, como C++, Python, Java o JavaScript, se parecen más al lenguaje humano y sirven para cualquier procesador. Los lenguajes por **bloques** (Scratch, mBlock, MakeCode) evitan los errores de escritura y son ideales para empezar.

Para que el ordenador ejecute un programa de alto nivel hay que traducirlo. Un **compilador** traduce todo el programa de una vez a código máquina antes de ejecutarlo (C++ en Arduino): el resultado es rápido, pero cada cambio exige volver a compilar. Un **intérprete** traduce y ejecuta línea a línea mientras el programa funciona (Python, JavaScript): es más cómodo para probar, aunque algo más lento. Cada lenguaje tiene su sintaxis, sus reglas de escritura: en C++ las instrucciones terminan en punto y coma y los bloques van entre llaves; en Python se usa la sangría. La lógica, en cambio, es la misma en todos: secuencia, decisión y repetición.




## 1.3 Variables y tipos de datos

Una **variable** es una «caja» con nombre donde el programa guarda un dato que puede cambiar: la lectura de un sensor, un contador, una puntuación. Cada variable tiene un **tipo**: número entero (`int`), decimal (`float`), verdadero/falso (`bool`) o texto (`String`). Con ellas se hacen **operaciones** aritméticas (+, –, *, /) y **comparaciones** (<, >, ==, !=).




## 1.4 Vectores (arrays)

Si un semáforo tiene tres LED en las patillas 11, 12 y 13, podríamos crear tres variables, pero es más práctico un **vector** o *array*: una lista de datos del mismo tipo con un solo nombre. En C++ se declara así: `int leds[3] = {11, 12, 13};`. Cada elemento se identifica por su **índice**, que empieza en `0`: `leds[0]` vale 11 y `leds[2]` vale 13. Intentar leer `leds[3]` es un error, porque ese elemento no existe, y en Arduino no avisa: devuelve un valor sin sentido.

La gran ventaja de los vectores es que se recorren con un bucle: `for (int i = 0; i < 3; i++) { pinMode(leds[i], OUTPUT); }` configura los tres LED con una sola instrucción repetida. También sirven para guardar varias lecturas de un sensor y calcular su media, lo que reduce el «ruido» de las medidas; para almacenar las notas de una melodía y sus duraciones; o para

guardar los ángulos de un brazo robótico en una secuencia de movimientos. Los textos, por cierto, también son vectores: de caracteres.

**int leds[3] = {11, 12, 13};**

| Índice | 0 | 1 | 2 |
|---|---|---|---|
| Valor | 11 | 12 | 13 |
| Acceso | leds[0] | leds[1] | leds[2] |
| LED | rojo | ámbar | verde |

## 1.5 Operadores

Los programas calculan y comparan con **operadores**. Los **aritméticos** hacen cuentas: + (suma), – (resta), * (multiplicación), / (división) y % (resto de la división; 7 % 3 da 1, útil para saber si un número es par). Los de **comparación** dan verdadero o falso: == (igual), != (distinto), <, >, <= y >=. Ojo: un solo = asigna un valor a una variable (x = 5), y dos == comparan (¿x == 5?). Los **lógicos** combinan condiciones: **Y** (&&, las dos deben cumplirse), **O** (||, basta una) y **NO** (!, invierte). Por ejemplo: «si hay poca luz Y hay alguien en la sala, enciende la lámpara».

| Operador | Significado | Ejemplo | Resultado |
|---|---|---|---|
| % | resto | 10 % 4 | 2 |
| == | igual a | 3 == 3 | verdadero |
| != | distinto de | 3 != 5 | verdadero |
| && | y | (5 > 3) && (2 > 4) | falso |
| \|\| | o | (5 > 3) \|\| (2 > 4) | verdadero |

**Operadores**

| Tipo | Operadores | Ejemplo |
|---|---|---|
| Aritméticos | + – * / % | 17 % 5 = 2 |
| Comparación | == != < > <= >= | x >= 10 |
| Lógicos | && \|\| ! | a && b |
| Asignación | = += ++ | i++ |

## 2.1 Estructuras de control

Los programas no siempre van en línea recta. El **condicional** `if … else` ejecuta unas instrucciones solo si se cumple una condición. Los **bucles** repiten instrucciones: `for` un número fijo de veces (parpadear 10 veces) y `while` mientras se cumpla una condición (esperar a que se pulse un botón). En Arduino, la función `loop()` ya es un bucle infinito.

```
for (int i = 0; i < 10; i++) {
  digitalWrite(13, HIGH);
  delay(500);
  digitalWrite(13, LOW);
  delay(500);
}
```

**repite 10 veces: enciende, espera, apaga**

## 2.2 Los bucles en detalle

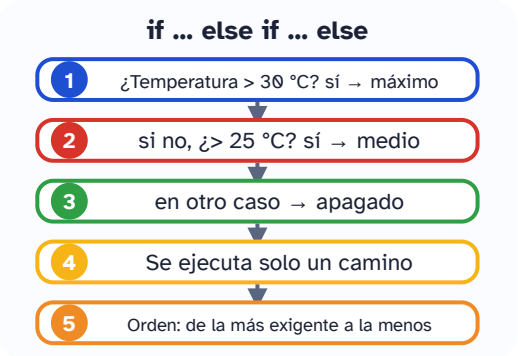
Un bucle **for** se usa cuando sabemos cuántas veces repetir. Tiene tres partes: un valor inicial del contador, una condición para seguir y un incremento. `for (int i = 0; i < 5; i++)` repite 5 veces, con i valiendo 0, 1, 2, 3 y 4. Un bucle **while** («mientras») repite mientras se cumpla una condición, sin saber de antemano cuántas veces: «mientras no se pulse el botón, haz parpadear el LED». Hay que asegurarse de que la condición acabe siendo falsa; si no, el programa queda atrapado en un **bucle infinito**. En Arduino, la función loop() es en sí misma un bucle infinito que repite el programa principal sin parar.




## 2.3 Condiciones múltiples: else if y switch

Muchas veces hay más de dos caminos. Con `if … else if … else` se encadenan condiciones que se comprueban en orden; en cuanto una se cumple, se ejecuta su bloque y se ignoran las demás. Por ejemplo, con un sensor de temperatura: si supera 30 °C, ventilador al máximo; *si no*, si supera 25 °C, ventilador a media velocidad; *en cualquier otro caso*, apagado. El orden importa: si se comprobara primero «más de 25», una temperatura de 32 °C entraría ahí y nunca llegaría a la condición de 30.

Cuando una variable entera puede tomar varios valores concretos, es más claro usar `switch`: `switch (modo) { case 1: … break; case 2: … break; default: … }`. Cada `case` es un valor posible y `default` recoge los demás. Hay que acordarse del `break`: sin él, el programa sigue ejecutando el caso siguiente. Es muy útil para menús, para los botones de un mando a distancia o para las máquinas de estados de un robot, en las que una variable `estado` indica qué está haciendo el sistema en cada momento.




## 2.4 Funciones

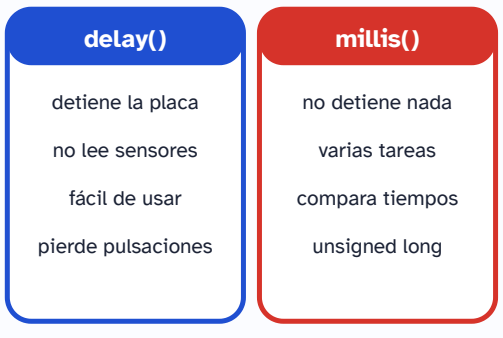
Una **función** es un bloque de código con nombre que hace una tarea concreta y se puede llamar muchas veces. Así el programa queda más corto, ordenado y fácil de corregir. Pueden recibir **parámetros** (datos de entrada) y devolver un resultado. Por ejemplo, `parpadear(3)` podría encender y apagar un LED tres veces. Todo programa de Arduino tiene dos funciones obligatorias: `setup()`, que se ejecuta una vez al empezar, y `loop()`, que se repite sin fin.

```
void setup() {                1 vez
  pinMode(13, OUTPUT);
}
void loop() {                 siempre
  parpadear(3);
  delay(2000);
}
```

## 2.5 Controlar el tiempo sin bloquear: millis()

La función `delay(1000)` detiene el programa un segundo. Es cómoda, pero mientras espera la placa no hace nada más: no lee pulsadores ni sensores. Si un LED parpadea con `delay` y a la vez queremos detectar un botón, muchas pulsaciones se perderán. La alternativa es `millis()`, que devuelve los milisegundos transcurridos desde que se encendió la placa, sin detener nada.

La técnica consiste en guardar el momento del último cambio y comprobar en cada vuelta del `loop` si ha pasado el tiempo deseado: `if (millis() - anterior >= 500) { anterior = millis(); cambiarLED(); }`. Así el `loop` sigue girando miles de veces por segundo y puede atender varias tareas «a la vez»: hacer parpadear dos LED a ritmos distintos, leer sensores y responder a botones. Como `millis()` crece mucho, se guarda en una variable de tipo `unsigned long`. Esta forma de programar, por eventos de tiempo en lugar de esperas, es la que usan los sistemas reales: una lavadora o un microondas nunca se quedan sordos mientras esperan.




## 3.1 Programar una placa: entradas y salidas

Una placa como **Arduino** o **micro:bit** tiene **pines** para conectar sensores y actuadores. Con `digitalRead()` se lee un pulsador (0 o 1) y con

`analogRead()` un sensor analógico, que da un valor entre `0` y `1023`. Con `digitalWrite()` se enciende o apaga un LED y con `analogWrite()` se regula su brillo o la velocidad de un motor. Se puede programar con **bloques** (mBlock, MakeCode) o con **texto** (lenguaje C++ de Arduino, Python).

```
int luz = analogRead(A0);
if (luz < 300) {
  digitalWrite(13, HIGH);
} else {
  digitalWrite(13, LOW);
}
```

**LDR en A0 · LED en el pin 13**

la farola automática en 6 líneas

## 3.2 Del programa a la placa

Para programar una placa como Arduino se escribe el código en un entorno de programación (el **IDE** de Arduino) o con bloques (mBlock, Arduinoblocks, Tinkercad). El programa se **compila**, es decir, se traduce al lenguaje que entiende el microcontrolador, y se **sube** a la placa por el cable USB. Todo programa tiene dos partes: **setup()**, que se ejecuta una vez al principio para configurar los pines como entradas o salidas, y **loop()**, que se repite sin fin. Para ver qué está pasando dentro se usa el monitor serie: con `Serial.println(valor)` la placa envía al ordenador el valor de una variable, lo que ayuda mucho a encontrar errores. Antes de montar, conviene simular el circuito en Tinkercad Circuits.




## 3.3 Comunicación serie

La placa y el ordenador se comunican por el cable USB mediante la **comunicación serie**, que envía los datos bit a bit, uno detrás de otro. En `setup()` se inicia con `Serial.begin(9600)`, donde `9600` es la velocidad en

**baudios** (bits por segundo), que debe coincidir en la placa y en el **monitor serie**. Con `Serial.print()` y `Serial.println()` la placa envía texto y valores al ordenador: es la mejor herramienta para ver qué está pasando dentro del programa.

La comunicación también funciona al revés: con `Serial.available()` se comprueba si han llegado datos y con `Serial.read()` se leen, de modo que se pueden enviar órdenes desde el teclado («e» para encender, «a» para apagar). El **trazador serie** dibuja gráficas en tiempo real de las lecturas, ideal para calibrar un sensor. Existen otros protocolos para conectar sensores y pantallas: **I2C**, que usa solo dos cables para muchos dispositivos, y **SPI**, más rápido. Con módulos Bluetooth o wifi (como la placa ESP32), la misma idea permite controlar un proyecto desde el móvil o enviar sus datos a Internet: el primer paso hacia el Internet de las cosas.




## 3.4 Un proyecto completo: el semáforo con pulsador

Veamos cómo se organiza un programa real. Un semáforo de peatones tiene tres LED para coches (rojo, ámbar y verde), dos para peatones y un pulsador. **Análisis**: normalmente los coches tienen verde; cuando un peatón pulsa, tras unos segundos el semáforo de coches pasa a ámbar y luego a rojo, los peatones tienen verde 10 segundos, su verde parpadea y todo vuelve al principio. **Diseño**: se dibuja el diagrama de flujo y se definen los estados: «coches», «ámbar», «peatones», «parpadeo».

**Programación**: las patillas se guardan en constantes con nombre (`const int ROJO = 13;`), así el código se entiende y, si se cambia un cable, solo hay que tocar una línea. Cada fase se escribe como una función (`fasePeatones()`), y el `loop` solo lee el pulsador y llama a la función que toca. **Pruebas**: se comprueba cada LED, después el pulsador mostrando su estado en el monitor serie y, al final,

la secuencia completa, cronometrando los tiempos. Esta forma de trabajar, analizar, diseñar, programar por partes y probar, es la misma que siguen los equipos profesionales de software.




## 3.5 Probar y depurar

Un programa casi nunca funciona a la primera. Los errores de **sintaxis** (un punto y coma olvidado, una llave sin cerrar, una mayúscula mal puesta: `digitalwrite` en vez de `digitalWrite`) los detecta el compilador y no deja subir el programa. Los errores de **lógica** son más difíciles: el programa funciona pero hace otra cosa, por ejemplo porque usamos > en vez de < o el pin equivocado. Para encontrarlos: probar con casos sencillos, mostrar las variables en el monitor serie, comprobar el circuito (muchos fallos son cables mal conectados o un LED al revés) y cambiar una sola cosa cada vez. Escribir **comentarios** (// así) en el código ayuda a entenderlo después.



## Las piezas de cualquier programa

| Variables | Condicional | Bucle | Función |
| --- | --- | --- | --- |
| guardan datos | decide | repite | agrupa tareas |
| `int luz = 512;` | `if (luz < 300)` | `for · while · loop` | `parpadear(3);` |

## Ejercicios

1. **¿Qué tipo de dato usarías para guardar: el número de vidas, la temperatura con decimales, si la puerta está abierta y el nombre del jugador?**

2. **Calcula: 17 % 5, 20 % 4 y 9 / 2 con enteros.**

3. **¿Es verdadera o falsa (x > 3) && (x < 10) si x = 12?**

4. **¿Cuántas veces se repite for (int i = 0; i < 8; i++)?**

5. **Escribe un bucle for que muestre los números del 10 al 1 hacia atrás.**

6. **¿Qué diferencia hay entre for y while?**

7. **Explica qué hacen setup() y loop() en Arduino.**

8. **Escribe una función que reciba un número y devuelva su doble.**

9. **Un LED no se enciende aunque el programa compila. Indica tres posibles causas.**

10. **Escribe en pseudocódigo un programa que encienda un LED si la lectura de una LDR es menor que 300 y lo apague si no.**

11. **¿Qué diferencia hay entre un compilador y un intérprete?**

12. **En int notas[5] = {7, 4, 9, 6, 8}; ¿cuánto vale notas[2]? ¿Y cuál es el último índice válido?**

13. **¿Para qué sirve Serial.begin(9600)?**

14. **¿Qué hace la instrucción break dentro de un switch?**

15. **¿Qué devuelve la función millis()?**

16. **Pon un ejemplo de lenguaje de bajo nivel, uno de alto nivel y uno por bloques.**

17. **Escribe un bucle que configure como salida las patillas guardadas en int leds[4] = {8, 9, 10, 11};**

18. **Escribe con else if: si la luz (0-1023) es menor que 200, «noche»; si es menor que 600, «nublado»; si no, «sol».**

19. **¿Por qué fallaría este orden: if (t > 25) {medio} else if (t > 30) {máximo}?**

20. **Escribe un switch que, según la variable modo (1, 2 o 3), encienda el LED rojo, ámbar o verde, y apague todo en otro caso.**

21. **Explica por qué un programa con delay(5000) dentro del loop no detecta bien un pulsador.**

22. **Calcula la media de 10 lecturas de A0 usando un bucle.**

23. **¿Por qué conviene usar const int ROJO = 13; en lugar de escribir 13 en todo el programa?**

24. **Escribe el programa que encienda el LED 13 si se recibe «e» por el monitor serie y lo apague si se recibe «a».**

25. **¿Qué ventaja tiene el protocolo I2C para conectar muchos sensores?**

26. **Traza este código y di qué muestra: int x = 0; for (int i = 1; i <= 4; i++) { x = x + i * i; } Serial.println(x);**

27. **Escribe un programa que haga parpadear el LED 13 cada 500 ms con millis(), sin delay.**

28. **Programa una melodía: guarda 4 notas en un vector y sus duraciones en otro y tócalas con tone() en la patilla 8.**

29. **Escribe una función que devuelva true si la temperatura está entre 18 y 24 °C (ambos incluidos).**

30. **Diseña la máquina de estados del semáforo con pulsador: estados, qué hace cada uno y qué provoca el cambio.**

31. **Dos LED deben parpadear a la vez, uno cada 300 ms y otro cada 700 ms. Explica por qué es imposible con delay y cómo hacerlo con millis.**

32. **Un sensor da lecturas que saltan mucho (ruido). Propón dos soluciones de programación.**

33. **Programa el riego del huerto: si la humedad (A0) es menor que 400, activa el relé (patilla 7) 5 segundos; luego espera una hora sin bloquear la lectura de un botón de parada.**

34. **¿Por qué la variable que guarda millis() debe ser unsigned long y no int?**

35. **Compara programar un proyecto con bloques y con código C++: ventajas de cada uno y cuándo elegirías cada opción.**

## Practica

1. **En un diagrama de flujo, ¿qué figura representa una decisión?**
   - **A)** Rectángulo
   - **B)** Triángulo
   - **C)** Rombo
   - **D)** Óvalo
2. **¿Qué es una variable?**
   - **A)** Un bucle infinito
   - **B)** Un error del programa
   - **C)** Un espacio con nombre donde se guarda un dato
   - **D)** Un tipo de sensor
3. **¿Qué estructura usarías para repetir algo 20 veces?**
   - **A)** if
   - **B)** for
   - **C)** else
   - **D)** while (false)
4. **En Arduino, ¿qué función se ejecuta una sola vez al encender la placa?**
   - **A)** loop()
   - **B)** setup()
   - **C)** pinMode()
   - **D)** delay()
5. **¿Entre qué valores da la lectura analogRead() en Arduino UNO?**
   - **A)** 0 y 1
   - **B)** 0 y 255
   - **C)** 0 y 1023
   - **D)** 0 y 100
6. **En programación, == significa...**
   - **A)** dividir
   - **B)** asignar
   - **C)** comparar si son iguales
   - **D)** sumar
7. **10 % 3 vale...**

**A)** 1
**B)** 3
**C)** 3,33
**D)** 0

8. **El operador && es verdadero si...**

**A)** se cumple al menos una condición
**B)** se cumplen las dos
**C)** ninguna es verdadera
**D)** no se cumple ninguna

9. **Un bucle que nunca termina se llama...**

**A)** bucle infinito
**B)** condicional
**C)** función
**D)** variable

10. **Para ver el valor de una variable desde la placa se usa...**

**A)** un potenciómetro
**B)** el monitor serie
**C)** un servomotor
**D)** un relé

11. **El lenguaje que entiende directamente el procesador es...**

**A)** Python
**B)** el código máquina
**C)** Scratch
**D)** C++

12. **Arduino se programa en C++, que es un lenguaje...**

**A)** interpretado
**B)** compilado
**C)** de bloques
**D)** de marcas

13. **En int v[4], el último elemento es...**

**A)** v[4]
**B)** v[3]
**C)** v[0]

**D)** v[5]

14. **En un switch, el caso que recoge los valores no previstos es...**

**A)** case 0

**B)** default

**C)** else

**D)** break

15. **La diferencia principal entre delay() y millis() es que...**

**A)** millis no detiene el programa

**B)** delay es más preciso

**C)** millis apaga la placa

**D)** no hay diferencia

16. **Para guardar el valor de millis() se usa...**

**A)** bool

**B)** char

**C)** unsigned long

**D)** float

17. **La velocidad de la comunicación serie se mide en...**

**A)** hercios

**B)** baudios

**C)** voltios

**D)** píxeles

18. **El protocolo que conecta muchos sensores con solo dos cables es...**

**A)** I2C

**B)** USB

**C)** HDMI

**D)** PWM

19. **En una cadena if ... else if, se ejecuta...**

**A)** todas las condiciones verdaderas

**B)** solo la primera condición verdadera

**C)** siempre el else

**D)** ninguna

20. **Usar const int ROJO = 13; sirve para...**

**A)** que el LED brille más

**B)** dar nombre claro a una patilla fija

**C)** acelerar el programa

**D)** crear un bucle

## Explícalo con tus palabras

Incluye: Qué es un algoritmo, con un ejemplo de la vida diaria; Los símbolos de un diagrama de flujo; Para qué sirven las variables y sus tipos; La diferencia entre un condicional y un bucle; Cómo lee un sensor y controla un LED una placa Arduino.

## Resumen




## Glosario

| | |
|---|---|
| **Algoritmo** | Secuencia ordenada y finita de pasos que resuelve un problema. |
| **Diagrama de flujo** | Representación gráfica de un algoritmo con símbolos normalizados. |
| **Código máquina** | Instrucciones en binario que ejecuta directamente el procesador. |
| **Lenguaje de alto nivel** | Lenguaje cercano al humano e independiente del procesador. |
| **Compilador** | Programa que traduce todo el código fuente a código máquina. |
| **Intérprete** | Programa que traduce y ejecuta el código línea a línea. |
| **Sintaxis** | Reglas de escritura de un lenguaje de programación. |
| **Variable** | Espacio con nombre donde se guarda un dato que puede cambiar. |
| **Constante** | Dato con nombre que no cambia durante el programa. |
| **Vector (array)** | Conjunto de datos del mismo tipo identificados por un índice. |
| **Condicional** | Estructura que ejecuta instrucciones solo si se cumple una condición. |
| **Bucle** | Estructura que repite instrucciones. |
| **Función** | Bloque de código con nombre que realiza una tarea. |

| | |
|---|---|
| **millis()** | Función de Arduino que da el tiempo desde el encendido. |
| **Comunicación serie** | Envío de datos bit a bit entre la placa y otro dispositivo. |
| **Baudio** | Unidad de velocidad de transmisión: bits por segundo. |
| **Depurar** | Buscar y corregir los errores de un programa. |
| **Máquina de estados** | Programa organizado en estados que cambian según las entradas. |

## Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

**¿Qué es un algoritmo?**

Una secuencia ordenada y finita de pasos para resolver un problema.

**Símbolo de decisión en un diagrama de flujo**

El rombo, con salidas sí y no.

**Diferencia entre for y while**

for repite un número fijo de veces; while repite mientras se cumpla una condición.

**Funciones obligatorias en Arduino**

setup() (una vez) y loop() (se repite siempre).

**¿Qué hace analogRead()?**

Lee un sensor analógico y devuelve un valor entre 0 y 1023.

**Diferencia entre = y ==**

= asigna un valor; == compara.

**Bucle while**

Repite mientras se cumpla una condición.

**Monitor serie**

Ventana donde la placa envía datos al ordenador; útil para depurar.

**Compilador e intérprete**

Traduce todo antes frente a traducir línea a línea.

**Vector (array)**

Lista de datos del mismo tipo; índices desde 0.

**millis()**

Tiempo desde el encendido; permite esperar sin bloquear.

**switch**

Elige entre varios casos de una variable; no olvidar break.

## Soluciones

1. **C)** El rombo tiene dos salidas: sí y no.
2. **C)** Su valor puede cambiar mientras el programa se ejecuta.
3. **B)** El bucle for repite un número fijo de veces.
4. **B)** setup() configura los pines; loop() se repite sin fin.
5. **C)** El conversor de 10 bits da 1024 valores posibles.
6. **C)** Un solo = asigna.
7. **A)** Es el resto de 10 / 3.
8. **B)** Es el Y lógico.
9. **A)** Su condición nunca es falsa.
10. **B)** Serial.println(valor).
11. **B)** Ceros y unos.
12. **B)** Se compila antes de subirlo a la placa.
13. **B)** Los índices van de 0 a 3.
14. **B)** Se ejecuta si ningún case coincide.
15. **A)** millis solo informa del tiempo transcurrido.
16. **C)** Es un número muy grande y siempre positivo.
17. **B)** Bits por segundo.
18. **A)** Cada dispositivo tiene su dirección.
19. **B)** Por eso importa el orden.
20. **B)** El código se lee mejor y se cambia en un solo sitio.

## Soluciones de los ejercicios

1. int, float, bool (boolean) y String.
2. 17 % 5 = 2; 20 % 4 = 0; 9 / 2 con enteros = 4 (se pierde la parte decimal).
3. Falsa: la primera es verdadera pero la segunda no, y && exige las dos.
4. 8 veces (i = 0 a 7).
5. for (int i = 10; i >= 1; i--) { Serial.println(i); }
6. for se usa cuando se sabe cuántas veces repetir; while repite mientras se cumpla una condición, sin saber de antemano cuántas veces.
7. setup() se ejecuta una vez al principio para configurar; loop() se repite indefinidamente con el programa principal.
8. int doble(int n) { return n * 2; }
9. El LED está conectado al revés, el número de pin del programa no coincide con el del circuito o falta configurar el pin como salida (también un cable suelto).
10. loop: luz = leer A0; si luz < 300 → LED encendido; si no → LED apagado.
11. El compilador traduce todo el programa a código máquina antes de ejecutarlo; el intérprete traduce y ejecuta línea a línea mientras funciona.
12. notas[2] vale 9 (los índices empiezan en 0). El último índice válido es 4.
13. Inicia la comunicación serie entre la placa y el ordenador a 9.600 baudios (bits por segundo).
14. Termina el caso actual y sale del switch; sin ella el programa seguiría ejecutando el caso siguiente.
15. Los milisegundos que han pasado desde que se encendió o reinició la placa.
16. Bajo nivel: ensamblador. Alto nivel: C++ o Python. Bloques: Scratch, mBlock o MakeCode.

17. ```
    for (int i = 0; i < 4; i++) {
      pinMode(leds[i], OUTPUT);
    }
    ```

18. ```
    if (luz < 200) {
      Serial.println("noche");
    } else if (luz < 600) {
      Serial.println("nublado");
    } else {
      Serial.println("sol");
    }
    ```

19. Una temperatura de 32 °C cumple ya t > 25 y entra en «medio»; la condición t > 30 nunca se alcanza. Hay que poner primero la más exigente.

20. ```
    switch (modo) {
      case 1: digitalWrite(ROJO, HIGH); break;
      case 2: digitalWrite(AMBAR, HIGH); break;
      case 3: digitalWrite(VERDE, HIGH); break;
    ```
    ```

```
    default: apagarTodo();
  }
```

21. Durante los 5 segundos de espera la placa no ejecuta nada más; si el pulsador se pulsa y se suelta en ese tiempo, nunca se lee.

22. ```
  long suma = 0;
  for (int i = 0; i < 10; i++) {
    suma = suma + analogRead(A0);
    delay(10);
  }
  int media = suma / 10;
  ```

23. El código se entiende mejor (ROJO dice qué es) y si se cambia el LED de patilla solo hay que modificar una línea.

24. ```
  void loop() {
    if (Serial.available() > 0) {
      char c = Serial.read();
      if (c == 'e') digitalWrite(13, HIGH);
      if (c == 'a') digitalWrite(13, LOW);
    }
  }
  ```

25. Solo necesita dos cables (datos y reloj) compartidos por todos los dispositivos, cada uno con su dirección, lo que ahorra patillas de la placa.
26. Suma 1 + 4 + 9 + 16 = 30. Muestra 30.

27. ```
  unsigned long anterior = 0;
  bool estado = false;
  void setup() {
    pinMode(13, OUTPUT);
  }
  void loop() {
    if (millis() - anterior >= 500) {
      anterior = millis();
      estado = !estado;
      digitalWrite(13, estado);
    }
  }
  ```

28. ```
  int notas[4] = {262, 294, 330, 349};
  int dur[4] = {300, 300, 300, 600};
  void setup() {
    for (int i = 0; i < 4; i++) {
  ```

```
      tone(8, notas[i], dur[i]);
      delay(dur[i] + 50);
    }
  }
  void loop() {}
```

29.
```
bool confortable(float t) {
  return t >= 18 && t <= 24;
}
```

30. COCHES: verde coches, rojo peatones; si se pulsa → ÁMBAR. ÁMBAR: 3 s → PEATONES. PEATONES: verde peatones 10 s → PARPADEO. PARPADEO: verde peatones parpadea 4 s → COCHES.
31. Con delay el programa espera a uno y bloquea al otro. Con millis se usan dos variables de tiempo (anterior1, anterior2): en cada vuelta del loop se comprueba por separado si han pasado 300 ms o 700 ms y se cambia solo el LED que toca.
32. Hacer la media de varias lecturas con un vector o un acumulador; usar histéresis en la decisión (dos umbrales distintos para encender y apagar) para que no cambie por pequeñas variaciones.
33. Usar millis(): si humedad < 400 y no se está regando, digitalWrite(7, HIGH) y guardar inicio = millis(); si millis() – inicio >= 5000, apagar el relé y guardar ultimoRiego = millis(); no volver a regar hasta que millis() – ultimoRiego >= 3600000. En cada vuelta se lee el botón y, si se pulsa, se apaga el relé.
34. En Arduino UNO un int llega solo hasta 32.767 (unos 32 segundos en milisegundos) y luego se desborda. Un unsigned long llega a más de 4.000 millones (unos 49 días).
35. Bloques: sin errores de sintaxis, visual y rápido para aprender y prototipos sencillos. Código: más potente y flexible, permite librerías y programas largos y es lo que se usa profesionalmente. Bloques para empezar o enseñar; código para proyectos complejos.